

Componentes Internos do Kubernetes

O que você aprenderá nesta página

Explicar, em termos gerais, como o Kubernetes opera.

Em vez de pensar no Kubernetes como algo completamente novo, percebi que compará-lo a um sistema operacional ajuda bastante. Não sou nenhum especialista em sistemas operacionais, mas todos nós os usamos.

O Kubernetes é uma camada sobre a qual executamos nossas aplicações. Ele pega os recursos que estão acessíveis a partir das camadas inferiores e gerencia nossas aplicações e recursos. Ele também fornece serviços, como DNS, para as aplicações. Com essa mentalidade de sistema operacional, também podemos tentar fazer o caminho inverso: você talvez já tenha utilizado um `cron` (ou o `task scheduler` no Windows) para agendar tarefas em lote (*batch jobs*), como salvar os backups de algumas aplicações. A mesma coisa pode ser feita no Kubernetes com CronJobs.

Agora que começaremos a falar sobre o funcionamento interno, obteremos novos conhecimentos a respeito do Kubernetes e seremos capazes de prever e resolver problemas que decorram de sua natureza.

Como esta seção é principalmente uma reiteração da documentação do Kubernetes, incluirei vários links para a documentação oficial. Apesar de estarmos falando sobre os componentes internos, não discutiremos como montar seu próprio cluster do zero manualmente. Se você deseja "sujar as mãos" e configurá-lo, deveria ler e completar *Kubernetes the Hard Way*, escrito por Kelsey Hightower.

Controladores (*Controllers*)

Os Controladores monitoram o estado do seu cluster e tentam sempre aproximar o estado atual do estado desejado. Quando você declara `X` réplicas de um Pod no seu `deployment.yaml`, um controlador chamado Replication Controller garantirá que isso seja mantido como verdade. Existem controladores específicos para várias responsabilidades.

Plano de Controle do Kubernetes (*Kubernetes Control Plane*)

O Plano de Controle do Kubernetes (*Control Plane*) é responsável pelo gerenciamento de todo o cluster do Kubernetes. É o principal componente de orquestração responsável por garantir que o estado desejado do cluster seja correspondente ao estado atual. O Control Plane toma decisões globais sobre o cluster (como, por exemplo, agendamento de tarefas) e também detecta e responde aos eventos do cluster (como o início de um novo Pod caso o número de réplicas de um deployment não esteja sendo satisfeito).

O Plano de Controle é formado por:

etcd: Um armazenamento chave-valor (*key-value storage*) que o Kubernetes usa para salvar e respaldar todos os dados do cluster. **kube-scheduler**: Decide em qual *node* (nó) um Pod deverá ser executado. **kube-controller-manager**: É o responsável por executar e controlar todos os controladores. **kube-apiserver**: Expõe o *Control Plane* do Kubernetes aos usuários por meio de uma API.

Existe também um componente extra chamado *cloud-controller-manager*, que permite vincular o cluster à API de um provedor de Nuvem. Se você quisesse montar seu próprio cluster na Hetzner, por exemplo, poderia utilizar o `hccloud-cloud-controller-manager` dentro do seu próprio cluster Kubernetes sendo hospedado através de máquinas virtuais (VMs) deles.

Componentes de Nó (Node Components)

Cada nó (node) possui alguns componentes com o intuito de manter os *Pods* funcionando:

kubelet: Garante que os contêineres estão executando as suas rotinas em um *Pod*. **kube-proxy:** É um proxy e um mantenedor de regras de rede. Permite conexões com a parte interna e externa do cluster, para que os Serviços operem conforme os usamos. Além disso, cada nó tem o chamado *Container Runtime*. Neste curso, estamos usando o **Docker** como tempo de execução.

Componentes Adicionais (Addons)

Apesar de tudo o que foi citado acima, o Kubernetes permite componentes adicionais (*Addons*), que utilizam os mesmos recursos do Kubernetes já discutidos e estendem suas funcionalidades. Você pode inspecionar quais recursos foram criados por essas extensões usando o comando no *namespace* kube-system:

```
$ kubectl -n kube-system get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/coredns-ccb96694c-t9rr2	1/1	Running	0	42h
pod/helm-install-traefik-crd-5446f	0/1	Completed	0	42h
pod/helm-install-traefik-p8c2h	0/1	Completed	2	42h
pod/local-path-provisioner-5cf85fd84d-s9x7p	1/1	Running	0	42h
pod/metrics-server-5985cbc9d7-hngw9	1/1	Running	0	42h
pod/svc-lb-traefik-68897c6e-bq9k7	2/2	Running	0	42h
pod/svc-lb-traefik-68897c6e-kw5p4	2/2	Running	0	42h
pod/svc-lb-traefik-68897c6e-qkc5c	2/2	Running	0	42h
pod/traefik-5d45fc8cc9-ws66x	1/1	Running	0	42h

NAME	PORT(S)	TYPE	CLUSTER-IP	EXTERNAL-IP	AGE
service/kube-dns	53/UDP, 53/TCP, 9153/TCP	ClusterIP	10.43.0.10	<none>	42h
service/metrics-server	443/TCP	ClusterIP	10.43.224.88	<none>	42h
service/traefik	80:32558/TCP, 443:32155/TCP	LoadBalancer	10.43.140.57	192.168.144.3, 192.168.144.4, 192.168.144.5	42h

Para ter uma visão abrangente sobre como cada parte conversa entre si, leia o artigo *what happens when k8s*, que explora o que ocorre de forma interna quando você executa um comando como `kubectl run nginx --image=nginx --replicas=3`, o que ilumina de verdade a "mágica" que atua por debaixo dos panos.

Auto-recuperação (Self-healing)

Anteriormente, conversamos brevemente a respeito da natureza de auto-recuperação do Kubernetes e sobre como *Pods* que são deletados podem ser recriados de maneira automática.

Mas o que ocorre se apagarmos ou derrubarmos um nó (node) contendo um *pod*? Primeiramente, faremos o *deploy* de um aplicativo web com *ingress*, e confirmaremos onde seu *pod* está sendo executado:

```
$ kubectl apply -f https://raw.githubusercontent.com/kubernetes-hy/material-example/master/app2/manifests/deployment.yaml \
-f https://raw.githubusercontent.com/kubernetes-hy/material-example/master/app2/manifests/ingress.yaml \
-f https://raw.githubusercontent.com/kubernetes-hy/material-example/master/app2/manifests/service.yaml
deployment.apps/hashresponse-dep created
ingress.extensions/dwk-material-ingress created
service/hashresponse-svc created

$ curl localhost:8081
9eafx3: 8k2deb
```

```
$ kubectl describe po hashresponse-dep-57bcc888d7-5gkc9 | grep 'Node:'
Node:          k3d-k3s-default-agent-1/172.30.0.2
```

Vemos aqui que ele está localizado em agent -1. Em seguida, faremos com que este nó saia da rede, tirando-o do ar usando um "pause" via docker:

```
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED
STATUS        PORTS                                     NAMES
5c43fe0a936e   rancher/k3d-proxy:v3.0.0           "/bin/sh -c nginx-pr..." 10 days ago
Up 2 hours    0.0.0.0:8081->80/tcp, 0.0.0.0:50207->6443/tcp  k3d-k3s-default-
server1b
fea775395132   rancher/k3s:latest                 "/bin/k3s agent"            10 days ago
Up 2 hours                                     k3d-k3s-default-agent-
1
70b68b040360   rancher/k3s:latest                 "/bin/k3s agent"            10 days ago
Up 2 hours    0.0.0.0:8082->30080/tcp              k3d-k3s-default-agent-
0
28cc6cef76ee   rancher/k3s:latest                 "/bin/k3s server --t..." 10 days ago
Up 2 hours                                     k3d-k3s-default-
server-0

$ docker pause k3d-k3s-default-agent-1
k3d-k3s-default-agent-1
```

Aguardaremos alguns minutos, e este deverá ser o estado que encontraremos:

```
$ kubectl get po
NAME                                READY   STATUS    RESTARTS   AGE
hashresponse-dep-57bcc888d7-5gkc9  1/1     Terminating    0          15m
hashresponse-dep-57bcc888d7-4klvg  1/1     Running         0          30s

$ curl localhost:8081
6xluh2: ta0ztp
```

E caso seja o único *control-plane* (Plano de Controle) que acabe deletado ou offline? Não ocorrerá nada de bom. Sendo um cluster local, ele se mostra um ponto de falha única (SPOF - Single Point of Failure). Confira a documentação do Kubernetes sobre Options for Highly Available topology para evitar que seu cluster inteiro fique inválido devido a uma simples falha de hardware.